

Assignment 8

1] Binary Search Tree with insertNode, removeNode and traversals

Program -

```
class BinarySearchTree
{
    constructor()
    {
        // root of a binary search tree
        this.root = null
    }
    getrootNode()
    {
        return this.root
    }

    insert(data)
    {
        var newNode = new Node(data)
        if(this.root == null)
            this.root = newNode
        else
            this.insertNode(this.root, newNode)
    }

    insertNode(node, newNode)
    {
        if(newNode.data < node.data)
        {
            // if left is null insert node here
            if(node.left == null)
                node.left = newNode
            else
                this.insertNode(node.left, newNode)
        }

        else
        {
            // if right is null insert node here
            if(node.right == null)
                node.right = newNode
            else
                this.insertNode(node.right, newNode)
        }
    }
    remove(data)
    {
        this.root = this.removeNode(this.root, data);
    }

    removeNode(node, key)
```

```

{
    if(node === null)
        return null;

    else if(key < node.data)
    {
        node.left = this.removeNode(node.left, key);
        return node;
    }
    else if(key > node.data)
    {
        node.right = this.removeNode(node.right, key);
        return node;
    }

    else
    {
        // deleting node with no children
        if(node.left === null && node.right === null)
        {
            node = null;
            return node;
        }

        // deleting node with one children
        if(node.left === null)
        {
            node = node.right;
            return node;
        }

        else if(node.right === null)
        {
            node = node.left;
            return node;
        }

        // Deleting node with two children
        // minimum node of the right subtree is stored in temp
        var temp = this.findMinNode(node.right);
        node.data = temp.data;
        node.right = this.removeNode(node.right, temp.data);
        return node;
    }
}

findMinNode(node)
{
    // if left of a node is null
    // then it must be minimum node
    if(node.left === null)
        return node;
    else
        return this.findMinNode(node.left);
}

inorder(node)
{
    //console.log(node.data)

```

```

        if(node != null)
        {
            this.inorder(node.left)
            console.log(node.data)
            this.inorder(node.right)
        }
    }
    preorder(node)
    {
        //console.log(node.data)
        if(node != null)
        {
            console.log(node.data)
            this.inorder(node.left)
            this.inorder(node.right)
        }
    }
    postorder(node)
    {
        //console.log(node.data)
        if(node != null)
        {
            this.inorder(node.left)
            this.inorder(node.right)
            console.log(node.data)
        }
    }
}

class Node
{
    constructor (data, left, right)
    {
        this.data = data
        this.left = null
        this.right = null
    }
}

const bst=new BinarySearchTree()
var r=new BinarySearchTree()
bst.insert(55)
bst.insert(10)
bst.insert(89)
bst.insert(62)
bst.insert(104)
bst.insert(60)
bst.insert(5)
bst.insert(-10)
bst.insert(20)
r=bst.getrootNode()
console.log("Inorder traversal of tree..")
bst.inorder(r)
console.log("Preorder traversal of tree..")
bst.preorder(r)
console.log("Postorder traversal of tree..")
bst.postorder(r)
bst.remove(89)

```

```
console.log("Inorder traversal of tree..")
bst.inorder(r)
bst.remove(-10)
console.log("Inorder traversal of tree..")
bst.inorder(r)
console.log("Inorder traversal of tree..")
bst.inorder(r)
bst.remove(10)
console.log("Inorder traversal of tree..")
bst.inorder(r)
```

Output -

```
Inorder traversal of tree..
-10
5
10
20
55
60
62
89
104
Preorder traversal of tree..
55
-10
5
10
20
60
62
89
104
Postorder traversal of tree..
-10
5
10
20
60
62
89
104
55
Inorder traversal of tree..
-10
5
10
20
55
60
62
104
Inorder traversal of tree..
5
10
20
55
60
```

```
62
104
Inorder traversal of tree..
5
10
20
55
60
62
104
Inorder traversal of tree..
5
20
55
60
62
104
```